

HomeGuardX v2.0

Modular FPGA Smart Home Controller

Complete Setup & User Guide

Vivado Project Setup • Synthesis & Implementation
Board Programming • Hardware Connections
Device Operation • Troubleshooting

Target Platform: Xilinx Artix-7 (Basys3 / Nexys A7)
Mohit Sai Vinnakota | VIT Vellore | ECE Department
February 2026 | Confidential

Table of Contents

Part I: Vivado Project Setup

- 1.1 Prerequisites
- 1.2 Creating the Vivado Project
- 1.3 Adding Source Files
- 1.4 Adding Constraints
- 1.5 Setting Include Paths

Part II: Simulation & Verification

- 2.1 Running Individual Testbenches
- 2.2 Full Integration Testbench
- 2.3 Interpreting Waveforms

Part III: Synthesis & Implementation

- 3.1 Running Synthesis
- 3.2 Reviewing Utilization Reports
- 3.3 Running Implementation
- 3.4 Timing Analysis
- 3.5 Generating the Bitstream

Part IV: Programming the FPGA

- 4.1 Connecting the Basys3 Board
- 4.2 Programming via Hardware Manager
- 4.3 Verifying the Bitstream Load

Part V: Hardware Connections

- 5.1 Pmod Connector Map
- 5.2 Connecting the Keypad (Pmod JA)
- 5.3 Connecting the OLED (Pmod JB)
- 5.4 Connecting RS-485 Link (Pmod JC)
- 5.5 Connecting PWM Fan & ADC (Pmod JD)
- 5.6 Connecting Sensors (PIR, Door, DHT22)

Part VI: Device Operation Guide

- 6.1 Switch & Button Layout
- 6.2 LED Status Indicators
- 6.3 Configuring Thresholds
- 6.4 Arming the Security System

- 6.5 Disarming via Keypad PIN
- 6.6 Changing the PIN Code
- 6.7 Understanding the OLED Display
- 6.8 Fan & Light Automatic Control
- 6.9 Dongle Authentication
- 6.10 Module Hot-Swap

Part VII: Troubleshooting

- 7.1 Common Synthesis Errors
- 7.2 Timing Failures
- 7.3 Hardware Debugging
- 7.4 Module Not Detected

Appendix

- A. HMCB Packet Reference
- B. Module ID Table
- C. Pin Assignment Quick Reference

Part I: Vivado Project Setup

1.1 Prerequisites

Before beginning, ensure you have the following installed and ready:

Requirement	Version	Notes
Vivado Design Suite	2023.1 or later	WebPACK edition is sufficient for Artix-7
Basys3 or Nexys A7 Board	—	XC7A35T or XC7A100T FPGA
USB Micro-B Cable	—	For JTAG programming and UART
Basys3 Board Files	Latest	Install via Vivado Board Manager

NOTE: The project targets XC7A35T-1CPG236C specifically. If using Nexys A7 (XC7A100T), update the part number in project settings and adjust pin constraints accordingly.

1.2 Creating the Vivado Project

Follow these steps to create a new Vivado project for HomeGuardX v2.0:

1

Launch Vivado

Open Vivado 2023.x. From the start screen, click **"Create Project"** (or File → Project → New).

2

Project Name & Location

Set project name to **HomeGuardX_v2**. Choose a working directory. Check "Create project subdirectory". Click Next.

3

Project Type

Select **"RTL Project"**. Check "Do not specify sources at this time" (we will add them manually). Click Next.

4

Select Part

In the Parts tab, search for **xc7a35tcp236-1**. Select it and click Next. Alternatively, go to the Boards tab and select "Basys3".

5

Finish

Review the summary and click **Finish**. The project will open in the Vivado IDE.

TIP: Alternatively, you can use the provided TCL scripts. Open Vivado in batch mode and run:
`vivado -mode batch -source scripts/synth.tcl`

1.3 Adding Source Files

Add the RTL source files to the project in the correct directory structure:

1

Add Sources

In the Flow Navigator (left panel), click **"Add Sources"** → "Add or create design sources" → Next.

2

Add Directories

Click **"Add Directories"**. Navigate to the HomeGuardX_v2/src/ folder and add all four subdirectories: core/, security/, environment/, io/. This imports all .v files.

3

Add Include File

Also add **src/hmcb_pkg.vh** as a header/include file. Vivado will detect it as a Verilog header.

4

Set Top Module

In the Sources panel, right-click **top_level_controller** and select "Set as Top". You should see the full hierarchy expand beneath it.

The source hierarchy should appear as follows:

```
top_level_controller (top)
+-- hmcb_arbiter
+-- module_registry
+-- timer_counter
+-- config_registers
+-- intruder_alert_controller
+-- pwm_fan_controller
+-- light_controller
+-- multi_sensor_hub
+-- oled_i2c_controller
+-- keypad_fsm
+-- pin_auth_controller
+-- prop_link_controller
```

1.4 Adding Constraints

1

Add Constraint Files

Click "Add Sources" → "Add or create constraints" → Next. Add both files from the constraints/ directory: **basys3_pins.xdc** and **timing.xdc**.

2 Verify

In the Sources panel, expand Constraints → constrs_1. Both XDC files should appear. The basys3_pins.xdc contains physical pin mappings; timing.xdc contains clock and I/O delay constraints.

NOTE: If you are not using all Pmod peripherals, comment out the unused pin assignments in basys3_pins.xdc to avoid DRC warnings. Unused I/O ports on the top module should be removed or tied off.

1.5 Setting Include Paths

The hmcb_pkg.vh file is included by most modules via a Verilog include directive. Vivado must know where to find it:

1 Open Settings

Go to **Settings** → **General** → **Verilog Options** (or right-click the fileset and select Properties).

2 Add Include Directory

Under "Verilog Include Files Search Paths", add the path to your **src/** directory (the folder containing hmcb_pkg.vh). Click OK.

3 Verify

Run a quick syntax check: right-click top_level_controller → "Elaborate Design". If it completes without errors about missing includes, the path is correct.

Part II: Simulation & Verification

2.1 Running Individual Testbenches

HomeGuardX v2.0 includes seven testbenches for unit and integration testing. To run a testbench:

- 1 Add Simulation Sources**
Click "Add Sources" → "Add or create simulation sources". Add all .v files from the **tb/** directory.
- 2 Select Testbench**
In the Sources panel, switch to the "Simulation Sources" view. Right-click the desired testbench (e.g., **tb_hmcb_arbiter**) and select "Set as Top".
- 3 Run Simulation**
In the Flow Navigator, click "**Run Simulation**" → "**Run Behavioral Simulation**". Vivado will compile and launch the simulator.
- 4 View Waveforms**
The waveform viewer opens automatically. Add signals of interest by dragging from the Objects panel. Press **Shift+F2** to run to the next breakpoint or end of simulation.

Available testbenches:

Testbench File	Tests	Expected Duration
tb_hmcb_arbiter.v	Bus arbitration, alarm priority	< 1 ms sim time
tb_pwm_fan.v	Duty cycle calculation, PWM output	< 1 ms
tb_keypad_fsm.v	4x4 scan, debounce, key decode	~5 ms
tb_pin_auth.v	Correct/wrong PIN, lockout logic	< 1 ms
tb_prop_link.v	LFSR auth, UART TX/RX framing	~1 ms
tb_oled_controller.v	I2C init, framebuffer refresh	~500 ms
tb_top_level_v2.v	Full integration: all modules	~1 sec

2.2 Full Integration Testbench

The **tb_top_level_v2** testbench exercises the complete system including boot-time presence detection, intruder alarm with arm/disarm, config register loading, and PWM fan observation. This is the most critical simulation to run before synthesis.

NOTE: The integration testbench simulates ~1 second of real time. At 100 MHz this is 100 million clock cycles. Expect the simulation to take 5-15 minutes depending on your machine. Use **run -all** in the Tcl console.

2.3 Interpreting Waveforms

Key signals to monitor during simulation:

Signal	Expected Behavior
hmcb_bus[31:0]	Active packets appear when hmcb_bus_valid is high. Check DEST_ID, CMD fields.
module_presence_reg[7:0]	Bits set after boot PING/PONG sequence (~200ms into simulation).
security_state[1:0]	Transitions: 0(IDLE) → 1(ARMED) → 2(TRIGGERED) → 3(ALARM) → 0(IDLE).
fan_duty[7:0]	Changes on tick_1hz. 0 when below threshold, scales linearly above.
pwm_fan_out	25 kHz square wave. Duty cycle should match fan_duty register.
auth_granted	Pulses high after correct 4-digit PIN entry (1-2-3-4).

Part III: Synthesis & Implementation

3.1 Running Synthesis

1

Start Synthesis

In the Flow Navigator, click **"Run Synthesis"**. Vivado will elaborate, optimize, and map the design to Artix-7 primitives. This takes 2-5 minutes.

2

Wait for Completion

A dialog appears when synthesis finishes. Select **"Open Synthesized Design"** to inspect the results.

3

Check Messages

Review the Messages panel for warnings. **Critical warnings** (red) must be resolved. Common warnings include undriven signals or unconstrained paths — these are usually safe if they are on unused I/O.

TIP: You can also run synthesis from the command line:

```
cd HomeGuardX_v2/scripts && vivado -mode batch -source synth.tcl
```

3.2 Reviewing Utilization Reports

After synthesis, check Reports → Report Utilization. The expected resource usage is extremely lean:

Resource	Used	Available	Utilization
LUTs	~830	20,800	~4.0%
Flip-Flops	~675	41,600	~1.6%
BRAM (18Kb)	3	50	6.0%
DSPs	1	90	1.1%

TIP: Over 95% of FPGA resources remain available for future modules such as VGA output, encryption accelerator, or a second zone.

3.3 Running Implementation

1

Launch Implementation

Click **"Run Implementation"** in the Flow Navigator. This performs placement, physical optimization, and routing. Takes 3-10 minutes.

2

Review Placement

After completion, select "Open Implemented Design" to view the physical floorplan. The design should be well-spread across the FPGA with no congestion.

3.4 Timing Analysis

Open the Timing Summary report (Reports → Timing → Report Timing Summary). Verify:

Check	Target	Action if Failed
Setup Slack (WNS)	≥ 0 ns	Optimize critical path or reduce clock speed
Hold Slack (WHS)	≥ 0 ns	Usually auto-fixed; add hold-fix buffers if negative
Pulse Width	No violations	Should pass easily at 100 MHz on Artix-7

TIP: At ~4% LUT utilization, timing closure at 100 MHz should be trivial. If WNS is negative, check the critical path in the timing report — it is likely in the PWM duty cycle division logic.

3.5 Generating the Bitstream

1

Generate Bitstream

Click **"Generate Bitstream"** in the Flow Navigator. Vivado runs implementation (if not already done) and produces the .bit file. Takes 1-3 minutes.

2

Locate the File

The bitstream is written to:

HomeGuardX_v2.runs/impl_1/top_level_controller.bit

Part IV: Programming the FPGA

4.1 Connecting the Basys3 Board

1

Connect USB

Connect the Basys3 to your PC via the Micro-USB port (top-left of board, labeled "PROG"). This provides both JTAG programming and power.

2

Power On

Set the power switch (bottom-left) to ON. The "DONE" LED should be off (no bitstream loaded yet). The "POWER" LED should be green.

3

Install Drivers

If Windows does not recognize the board, install the **Digilent Adept Runtime** and FTDI drivers. Vivado usually includes these.

4.2 Programming via Hardware Manager

1

Open Hardware Manager

In Vivado, click "**Open Hardware Manager**" at the bottom of the Flow Navigator, then click "**Open Target**" → "**Auto Connect**".

2

Detect Device

Vivado detects the XC7A35T. It appears in the Hardware panel as **xc7a35t_0**.

3

Program Device

Right-click the device → "**Program Device**". In the dialog, the bitstream file should be pre-filled. Verify the path and click "**Program**".

4

Verify

The "DONE" LED on the Basys3 turns green when programming succeeds. LEDs should begin showing module presence status within 1 second.

NOTE: The bitstream is volatile — it is lost when power is removed. To make it persistent, write it to the on-board SPI flash via Hardware Manager → Add Configuration Memory Device → select s25fl032 → Program.

Part V: Hardware Connections

5.1 Pmod Connector Map

HomeGuardX v2.0 uses four Pmod connectors for external peripherals. Each connector provides 12 pins (8 I/O + VCC + GND).

Pmod Header	Connected Peripheral	Connector Type
JA (pins 1-4, 7-10)	4x4 Matrix Keypad	8-pin ribbon or direct wire
JB (pins 1-2)	SSD1306 128x64 OLED (I2C)	4-pin (VCC, GND, SCL, SDA)
JC (pins 1-2)	RS-485 Transceiver + Dongle	4-pin (VCC, GND, A, B)
JD (pins 1-2, 3-6)	PWM Fan + ADC for sensors	Multi-pin

WARNING: All Pmod I/O operates at 3.3V LVCMOS. Do not connect 5V peripherals directly — use level shifters where needed (e.g., RS-485 transceiver boards often include level shifting).

5.2 Connecting the Keypad (Pmod JA)

Connect a standard 4x4 matrix keypad to Pmod JA. The keypad has 8 pins: 4 row outputs and 4 column inputs.

Keypad Pin	Function	FPGA Pin	Pmod JA Pin
R0	Row 0 drive	J1	JA1
R1	Row 1 drive	L2	JA2
R2	Row 2 drive	J2	JA3
R3	Row 3 drive	G2	JA4
C0	Column 0 read	H1	JA7
C1	Column 1 read	K2	JA8
C2	Column 2 read	H2	JA9
C3	Column 3 read	G3	JA10

TIP: Column inputs have internal pull-downs enabled in the XDC. If your keypad module has no pull-down resistors, the internal pull-downs should suffice for proper operation.

5.3 Connecting the OLED Display (Pmod JB)

Connect an SSD1306 128x64 OLED module (I2C interface, address 0x3C) to Pmod JB:

OLED Pin	FPGA Pin	Pmod JB Pin	Notes
VCC	3.3V	VCC	Power from Pmod header
GND	GND	GND	Ground
SCL	A14	JB1	I2C clock (400 kHz, pull-up enabled)
SDA	A15	JB2	I2C data (bidirectional, pull-up enabled)

5.4 Connecting RS-485 Link (Pmod JC)

Connect an RS-485 transceiver module (e.g., MAX485 breakout) to Pmod JC for the proprietary dongle link:

Signal	FPGA Pin	Pmod JC Pin	RS-485 Module Pin
TX / Driver Input	K17	JC1	DI (Driver Input)
RX / Receiver Out	M18	JC2	RO (Receiver Output)
VCC	3.3V	VCC	VCC
GND	GND	GND	GND

NOTE: Tie the DE (Driver Enable) and RE (Receiver Enable) pins of the MAX485 to the rs485_tx_en signal for proper half-duplex operation. If your transceiver board does not expose these, check if it has auto-direction control.

5.5 Connecting PWM Fan & ADC (Pmod JD)

Pmod JD hosts the fan PWM output, TACH feedback, and the serial ADC interface for analog sensors:

Signal	FPGA Pin	Pmod JD Pin
PWM Fan Output	H4	JD1
Fan TACH Input	H1	JD2
ADC CS_N	G1	JD3

Signal	FPGA Pin	Pmod JD Pin
ADC SCLK	G3	JD4
ADC MISO	H2	JD7
ADC CH_SEL[0]	G4	JD8
ADC CH_SEL[1]	G2	JD9

5.6 Connecting Sensors (PIR, Door, DHT22)

The PIR motion sensor, door contact sensor, and DHT22 temperature/humidity sensor connect directly to FPGA GPIO pins:

Sensor	FPGA Pin	Notes
PIR Motion (HC-SR501)	A14	Active high output; 3.3V compatible
Door Contact Switch	A16	Normally closed; open = door open = 1
DHT22 Data	F4	Bidirectional; 4.7k pull-up to 3.3V
Room Lights Relay	B18	Drive a transistor/relay for mains switching
Buzzer	A18	Active buzzer; drive directly or via transistor

Part VI: Device Operation Guide

6.1 Switch & Button Layout

The Basys3 has 16 switches (SW0-SW15) and 5 push buttons. HomeGuardX v2.0 maps them as follows:

Control	Assignment	Function
SW[7:0]	Data input	8-bit value for threshold configuration
SW[10:8]	Register select	Which threshold register to write (0-5)
SW[15]	Arm switch	UP = Armed, DOWN = Disarmed
BTN[0] (Center)	Load button	Press to write SW[7:0] into selected register
BTN[1-4]	Reserved	Available for future use

6.2 LED Status Indicators

LED	Indicator	Meaning
LED[0]	Room Lights	ON = Lights active (ambient light below threshold)
LED[1]	Alarm LED	Blinks during TRIGGERED, blinks during ALARM
LED[2]	Alarm Active	Solid ON = Alarm is sounding
LED[3]	Auth Granted	Pulses = Correct PIN entered
LED[4]	Link Auth	ON = RS-485 dongle authenticated
LED[5]	Dongle Present	ON = Dongle detected on RS-485 bus
LED[6]	Keypad Locked	ON = Keypad locked (3 failed PIN attempts)
LED[7]	Overheat	ON = Fan at 100% for 3+ seconds
LED[13:8]	Module Presence	Each bit = 1 optional module detected
LED[14]	Heartbeat	Blinks at ~2 Hz to confirm system is running
LED[15]	Arm Status	Mirrors SW[15]: ON = System Armed

6.3 Configuring Thresholds

The system has 6 configurable threshold registers that control automatic behaviors:

Register	SW[10:8]	Default	Unit	Controls
Temp Threshold	000	30	Degrees C	Fan turns on above this temperature
Light Threshold	001	80	ADC (0-255)	Lights on below this ambient level
Humidity Thresh	010	70	% RH	Humidity alarm above this level
Gas Threshold	011	150	ADC (0-255)	Gas/smoke alarm trigger level
CO Threshold	100	100	ADC (0-255)	Carbon monoxide alarm trigger
Temp Headroom	101	20	Degrees C	Fan PWM full-scale range above threshold

To change a threshold:

1

Set Value

Set SW[7:0] to the desired 8-bit value (binary on switches). For example, to set 35, flip SW5, SW1, and SW0 ON (binary 00100011).

2

Select Register

Set SW[10:8] to the register number. For example, 000 for temperature threshold.

3

Press Load

Press the center button (BTN[0]). The value is latched into the selected register immediately.

6.4 Arming the Security System

To arm the intruder detection system:

1

Flip SW[15] UP

The system transitions from IDLE to ARMED. LED[15] turns on.

2

System Is Monitoring

The PIR motion sensor and door contact sensor are now active. Any detection triggers a 10-second grace period (LED[1] blinks slowly).

3**Grace Period**

During the 10-second grace period, you can disarm via SW[15] or by entering the correct PIN on the keypad.

4**Full Alarm**

If not disarmed within 10 seconds, the system enters ALARM state: LED[1] blinks fast, LED[2] goes solid, and the buzzer sounds continuously.

6.5 Disarming via Keypad PIN

The default PIN is **1234**. To disarm the alarm or cancel a triggered state:

1**Enter PIN**

On the 4x4 keypad, press: **1, 2, 3, 4** (one digit at a time, waiting for each to register).

2**Auth Granted**

LED[3] pulses briefly. If in ALARM or TRIGGERED state, the system returns to IDLE and the buzzer stops.

3**Failed Attempt**

If the PIN is wrong, the system stays in its current state. After **3 failed attempts within 60 seconds**, the keypad is locked for 5 minutes (LED[6] ON).

WARNING: The lockout timer runs on the hardware tick_1hz clock. During lockout, all keypad input is ignored. Wait for LED[6] to turn off before retrying.

6.6 Changing the PIN Code

PIN changes are only allowed when the system is in the **DISARMED** state (SW[15] = DOWN):

1**Hold # for 3 Seconds**

Press and hold the # key on the keypad for 3 seconds to enter PIN change mode.

2**Enter Current PIN**

Type the current 4-digit PIN (default: 1234).

3**Enter New PIN**

Type the new 4-digit PIN.

4**Confirm New PIN**

Type the new PIN again. If both entries match, the PIN is updated in BRAM.

NOTE: The PIN is stored in BRAM and is volatile — it resets to 1234 on power cycle. To make it persistent, write the desired default PIN in the `pin_auth_controller.v` source (`stored_pin` initial value) and regenerate the bitstream.

6.7 Understanding the OLED Display

The SSD1306 OLED shows an 8-row live status dashboard:

Row	Content	Example
Row 0	Static header	HomeGuardX v2.0
Row 1	Security state	ARMED / DISARMED / ALARM
Row 2	Temperature & humidity	Temp: 28C Hum: 55%
Row 3	Gas & CO status	Gas: OK CO: OK
Row 4	Fan duty & lights	Fan: 40% Light: ON
Row 5	Authentication status	Auth: LOCKED / GRANTED
Row 6	Module presence bits	Modules: 111111
Row 7	Scrolling alerts	Any active alarm messages

The display refreshes at 4 Hz (250ms per frame). Sensor data is updated on the 1 Hz tick.

6.8 Fan & Light Automatic Control

Fan Control: The PWM fan activates automatically when the temperature reading exceeds the temperature threshold. The fan speed scales linearly from 0% to 100% over the headroom range. For example, with `threshold=30` and `headroom=20`: at 30C the fan is off; at 40C it runs at ~50%; at 50C+ it runs at 100%. If the fan stays at 100% for 3 consecutive seconds, an overheat alarm is raised (LED[7]).

Light Control: Room lights turn on automatically when ambient light drops below (`light_threshold - 4`) and turn off when it rises above (`light_threshold + 4`). The 4-count hysteresis band prevents flickering at the threshold boundary.

6.9 Dongle Authentication

External control of the system requires a physical authentication dongle connected via the RS-485 link:

1

Connect Dongle

Plug the dongle into the keyed 4-pin connector on the RS-485 transceiver board.

2

Auth Initiated

The core sends an LFSR challenge over HMCB to the link controller, which forwards it over RS-485 to the dongle.

3

Challenge-Response

The dongle computes: $\text{response} = \text{XOR}(\text{challenge}, \text{Secret}[31:0]) \text{ XOR } \text{Secret}[63:32]$ and returns it.

4

Verification

If the response matches, LED[4] (Link Auth) and LED[5] (Dongle Present) light up. The session is valid for 60 seconds before re-authentication.

6.10 Module Hot-Swap

HomeGuardX v2.0 supports hot-swapping of optional plug-in modules. The module registry broadcasts a PING every 10 seconds and updates the presence register based on which modules respond with PONG.

When a module is disconnected mid-operation, the system detects its absence within 10 seconds, broadcasts a MODULE_LOST fault, and falls back to safe defaults (fan off, lights off, alarms suppressed for that sensor). You can monitor module presence on LED[13:8]:

LED	Bit	Module
LED[8]	Bit 0	PWM Fan Controller (0x5)
LED[9]	Bit 1	Light Controller (0x6)
LED[10]	Bit 2	Multi-Sensor Hub (0x7)
LED[11]	Bit 3	OLED Display (0x8)
LED[12]	Bit 4	Keypad + Auth (0x9)
LED[13]	Bit 5	Proprietary Link (0xA)

Part VII: Troubleshooting

7.1 Common Synthesis Errors

Error	Cause	Solution
Cannot find hmc_b_pkg.vh	Include path not set	Add src/ to Verilog include paths in project settings
Multiple drivers on net	Two modules driving same signal	Check HMCB TX wiring in top_level_controller.v
Port mismatch	Module port name changed	Ensure all instantiations match module declarations
Undeclared identifier	Missing <code>`include</code> directive	Add <code>`include "hmc_b_pkg.vh"</code> to affected module

7.2 Timing Failures

If timing analysis reports negative slack:

1

Identify Critical Path

Open the timing report and locate the path with worst negative slack. It usually involves the division operation in pwm_fan_controller duty cycle calculation.

2

Pipeline the Logic

Insert a register stage in the critical path to break it into two clock cycles. For the PWM division, pre-compute the duty on one tick and apply it on the next.

3

Reduce Clock Speed

As a last resort, reduce the system clock. The design easily runs at 50 MHz if 100 MHz proves difficult (unlikely at 4% utilization).

7.3 Hardware Debugging

If the design loads but behaves unexpectedly:

Symptom	Check
No LEDs active after programming	Verify reset polarity. The rst_n input is active-low; hold BTN (center) during test.

Symptom	Check
Module presence LEDs all off	Check that MODULE_PRESENT=1 for compiled modules. Verify Pmod connections.
Keypad not responding	Verify row/column pin mapping. Use an oscilloscope to observe kp_row toggling.
OLED blank	Check I2C pull-ups (4.7k to 3.3V). Verify address 0x3C. Monitor SCL with scope.
Fan always off	Check temp_reading vs threshold. If sensor hub is absent, temp defaults to 25C.
Buzzer stuck on	Disarm with SW[15]=DOWN. If persistent, check alarm_active signal with ILA.

TIP: For in-depth debugging, use Vivado's Integrated Logic Analyzer (ILA). Insert an ILA core on the HMCB bus to capture packet traffic in real time. Mark hmcb_bus and hmcb_bus_valid for debug in synthesis settings.

7.4 Module Not Detected

If a specific module does not appear in the presence register:

- 1 Check MODULE_PRESENT**
 Verify that the module was compiled with MODULE_PRESENT=1 in the top-level instantiation.
- 2 Check HMCB Wiring**
 In top_level_controller.v, confirm the module's hmcb_tx and hmcb_tx_valid are connected to the arbiter.
- 3 Check Module ID**
 Ensure the module responds to PING with the correct Module ID in DATA_L of the PONG packet.
- 4 Simulate**
 Run tb_top_level_v2 and observe the PING/PONG exchange in the first 200ms of simulation.

Appendix

A. HMCB Packet Reference

Every HMCB transaction is a single 32-bit word:

Bits	Field	Width	Description
[31:28]	DEST_ID	4 bits	Destination Module ID (0x0 = broadcast)
[27:24]	SRC_ID	4 bits	Source Module ID
[23:16]	CMD	8 bits	Command opcode
[15:8]	DATA_H	8 bits	Data high byte
[7:0]	DATA_L	8 bits	Data low byte / status

Command opcodes:

CMD	Name	Description
0x00	PING	Broadcast: presence query
0x01	PONG	Presence acknowledgement
0x10	DATA_REPORT	Sensor/status value report
0x11	THRESHOLD_SET	Write a threshold register
0x20	ALARM_RAISE	Module signals alarm condition
0x21	ALARM_CLEAR	Core clears alarm
0x30	AUTH_CHALLENGE	LFSR challenge to link controller
0x31	AUTH_RESPONSE	Dongle response from link controller
0x32	AUTH_GRANT	Successful authentication notification
0x40	DISPLAY_UPDATE	Core sends data to OLED framebuffer
0xFF	FAULT	Critical fault broadcast

B. Module ID Table

ID	Module	Type	Optional?
0x0	BROADCAST	—	N/A
0x1	Core Controller	Onboard	No
0x2	Timer Counter	Onboard	No
0x3	Config Registers	Onboard	No
0x4	Intruder Alert FSM	Onboard	No
0x5	PWM Fan Controller	Plug-in A	Yes
0x6	Light Controller	Plug-in B	Yes
0x7	Multi-Sensor Hub	Plug-in C	Yes
0x8	OLED Controller	Plug-in D	Yes
0x9	Keypad + Auth FSM	Plug-in E	Yes
0xA	Proprietary Link	Plug-in F	Yes
0xB-0xF	Reserved	Future	—

C. Pin Assignment Quick Reference

Function	FPGA Pin	Board Location
System Clock (100 MHz)	W5	Onboard oscillator
Reset (active low)	U18	Center button
Switches SW[15:0]	V17...R2	16 slide switches
LEDs LED[15:0]	U16...L1	16 green LEDs
Keypad Rows[3:0]	J1, L2, J2, G2	Pmod JA upper
Keypad Cols[3:0]	H1, K2, H2, G3	Pmod JA lower
OLED SCL, SDA	A14, A15	Pmod JB pins 1-2
RS-485 A, B	K17, M18	Pmod JC pins 1-2
PWM Fan Out	H4	Pmod JD pin 1
Buzzer	A18	GPIO
Room Lights	B18	GPIO

HomeGuardX v2.0 | Pre-Patent Confidential | Mohit Sai Vinnakota | VIT Vellore | 2026